

Sling Ecommerce — Customer API Documentation

Customer-facing storefront API — registration, login, catalog browsing, profile, and orders

Base URL & Routing

Every endpoint is scoped to a shop by its `{company}` slug, as a URL segment (not a query parameter):

```
{base_url}/users/{company}/...
```

Example (local dev): `http://127.0.0.1:8000/users/vasantham/customers/register`

Company slug validation: every request first passes through a middleware that checks `{company}` against an active shop's `slug_name`. If it doesn't match any shop, every endpoint returns:

```
HTTP 404
Invalid Company.
```

Always confirm the slug is correct before debugging anything else — most early integration errors trace back to this.

Authentication

Two tiers of endpoints:

- **Public** — Register, Login, Categories, Sub-categories, Products. No token needed.
- **Authenticated** requires token — Profile, Orders. Requires a Bearer token obtained from Login.

```
Authorization: Bearer {token}
Accept: application/json
Content-Type: application/json
```

The token is a Laravel Sanctum personal access token, returned as `auth_token` in the Login response. It does not expire on a fixed schedule — treat it as valid until the user logs out or it's revoked server-side.

Standard Response Envelope

Every response — success or failure — uses the same shape:

```
// Success
{
  "code": 200,
  "message": "Human-readable success message",
  "success": true,
  "data": { ... }
}

// Error / validation failure
{
  "code": 400,
  "message": "Failed to register",
  "success": false,
  "data": null,
}
```

```
"errors": ["Validation or business-logic error message(s)"]
}
```

HTTP CODE	MEANING
200	Success
400	Validation failed, or a business rule was violated (e.g. "User already exists")
401	Missing/invalid/expired Bearer token on an authenticated endpoint
404	Company slug not found (see note above)
422	Order validation failed (cart/items malformed)
500	Server error — check storage/logs/laravel.log for the real cause

1. Customer Registration & Login

Register a new customer

POST /users/{company}/customers/register

Request body (JSON or form-urlencoded):

FIELD	REQUIRED	RULES
name	Yes	string, max 50
phone	Yes	10 digits, must differ from alt_phone
password	Yes	6–20 chars, must include upper, lower, digit, and one of @\$!%*#?&; requires password_confirmation to match
address	Yes	string, max 200
email	No	valid email
alt_phone	No	10 digits, must differ from phone
pincode	No	6 digits
gst	No	standard GSTIN format
gender, dob	No	accepted, not validated

```
// Sample request
POST /users/vasantham/customers/register
Content-Type: application/json

{
  "name": "Ravi Kumar",
  "phone": "9876543210",
  "password": "Test@123",
  "password_confirmation": "Test@123",
  "address": "12 Anna Salai, Chennai",
  "email": "ravi@example.com",
  "pincode": "600001"
}
```

```
// Success (200)
{
  "code": 200,
  "message": "Successfully registered",
  "success": true,
  "data": "Success"
}

// Already registered (400)
{
  "code": 400,
  "message": "Failed to register",
  "success": false,
  "data": null,
  "errors": "User already exist."
}
```

Login

POST

/users/{company}/customers/login

FIELD	REQUIRED
phone	Yes
password	Yes

```
// Sample request
POST /users/vasantham/customers/login
{ "phone": "9876543210", "password": "Test@123" }

// Success (200) – save auth_token for all subsequent requests
{
  "code": 200,
  "message": "Successfully Logged in",
  "success": true,
  "data": {
    "id": 57,
    "name": "Ravi Kumar",
    "role_id": 4,
    "customer_id": 12,
    "auth_token": "3|aBcD1234...",
    "customer": { "id": 12, "name": "Ravi Kumar", "phone": "9876543210", "address": "..." }
  }
}
```

Possible errors: account not active (email not verified), locked, deleted, or invalid credentials — each returns 400 with a specific message.

2. Catalog Browsing public — no token

All three endpoints below are **GET** requests — every value shown is a query-string parameter appended to the URL, not a request body.

List categories

GET

/users/{company}/categories

QUERY PARAM	TYPE	REQUIRED	DESCRIPTION
name	string	No	Filter categories by name (partial match)

```
// Sample request
GET /users/vasantham/categories?name=applian

// Sample response
{
  "code": 200,
  "message": "Category retrieved successfully.",
  "success": true,
  "data": [
    { "id": 1, "name": "Home Appliances", "image": "http://.../storage/categories/1.png",
      "sub_categories": [ { "id": 4, "name": "Fans", "category_id": 1 } ] }
  ]
}
```

List sub-categories

GET /users/{company}/sub_categories

QUERY PARAM	TYPE	REQUIRED	DESCRIPTION
category	integer	Yes	Category ID to filter sub-categories by
name	string	No	Filter by name (partial match)

```
// Sample request
GET /users/vasantham/sub_categories?category=1&name=fan

// Sample response
{
  "code": 200,
  "message": "Sub Category retrieved successfully.",
  "success": true,
  "data": [
    { "id": 4, "name": "Fans", "category_id": 1, "image": "http://.../storage/sub_categories/4.png",
      "category": { "id": 1, "name": "Home Appliances" } }
  ]
}
```

List products (in-stock catalog)

GET /users/{company}/products

QUERY PARAM	TYPE	REQUIRED	DESCRIPTION
product	string	No	Search by product, category, or sub-category name
stock_in	1 or 0	No	Pass 1 to show only items with quantity > 0
page	integer	No	Pagination page number (10 per page)

```
// Sample request
GET /users/vasantham/products?product=fan&stock_in=1&page=1

// Sample response (truncated)
{
```

```

"code": 200,
"message": "Stock retrieved successfully.",
"success": true,
"data": {
  "stocks": {
    "data": [
      { "id": 101, "quantity": 12,
        "product": { "id": 3, "name": "Ceiling Fan", "price": 2499, "image": "http://...",
          "category": { "id": 1, "name": "Home Appliances" },
          "sub_category": { "id": 4, "name": "Fans" } } }
    ],
    "current_page": 1, "last_page": 3, "per_page": 10, "total": 24
  },
  "categories": [ { "id": 1, "name": "Home Appliances" } ]
}
}

```

3. Profile requires token

View profile

GET `/users/{company}/profile/view`

No query parameters — the profile returned is always the logged-in customer's own (resolved from the Bearer token).

```

// Sample response
{
  "code": 200,
  "message": "Profile return successfully",
  "success": true,
  "data": {
    "id": 57, "name": "Ravi Kumar", "phone": "9876543210", "email": "ravi@example.com",
    "logo": "http://.../no-image-icon.svg",
    "customer": { "id": 12, "address": "12 Anna Salai, Chennai", "pincode": "600001" }
  }
}

```

Update profile

POST `/users/{company}/profile/update`

FIELD	REQUIRED	NOTES
name	Yes	max 50
address	Yes	max 200
email, alt_phone, pincode, gst, dob	No	same rules as Register
password	No	only updated if provided; same complexity rule as Register, needs password_confirmation
profile_photo	No	multipart file upload — jpg/jpeg/png/gif/webp, max 2MB

```

// Sample response
{ "code": 200, "message": "Profile updated successfully.", "success": true, "data": "Success" }

```

4. Orders

requires token

Place an order

POST

/users/{company}/orders/store

FIELD	REQUIRED	NOTES
cart	Yes	array, at least 1 item
cart[].product_id	Yes	must exist in products
cart[].qty	Yes	numeric, min 1
cart[].price	Yes	numeric, min 0
cart[].tax_amount	No	numeric
cart[].variation_id	No	size/colour variation, must exist in stock_variations
cart[].imeis	No	array of IMEI strings (electronics)
customer	No	customer ID (defaults to logged-in customer if omitted)
discount	No	numeric, min 0
billing_customer	No	object: billing_name, billing_phone, billing_address, billing_pincode

The shop must have an active Bill Setup configured (bill numbering prefix) or this returns 400 "No active bill setup found." — check with the shop admin if you hit this.

```
// Sample request
POST /users/vasantham/orders/store
{
  "customer": 12,
  "discount": 0,
  "cart": [
    { "product_id": 3, "qty": 2, "price": 2499, "tax_amount": 450 }
  ],
  "billing_customer": {
    "billing_name": "Ravi Kumar",
    "billing_phone": "9876543210",
    "billing_address": "12 Anna Salai, Chennai",
    "billing_pincode": "600001"
  }
}
```

```
// Success (200) – data is the new order's ID
{ "code": 200, "message": "Order saved successfully", "success": true, "data": 88 }
```

List my orders

GET

/users/{company}/orders/list

QUERY PARAM	TYPE	REQUIRED	DESCRIPTION
order	string	No	Search by bill ID, customer name/phone/GST
page	integer	No	Pagination page (10 per page)

Only returns the logged-in customer's own online orders for this shop — scoped automatically via the Bearer token.

```
// Sample request
GET /users/vasantham/orders/list?order=BR01&page=1
```

View one order

```
GET /users/{company}/orders/view/{id}
```

{id} is a path segment (the order's numeric ID), not a query parameter.

```
// Sample request
GET /users/vasantham/orders/view/88

// Sample response
{
  "code": 200,
  "message": "Order retrieved successfully.",
  "success": true,
  "data": {
    "order": { "id": 88, "bill_id": "BR01", "bill_amount": 5448, "status": 0, ... },
    "order_details": [
      { "product_id": 3, "name": "Ceiling Fan", "quantity": 2, "price": 2499, "tax_amount": 450 }
    ],
    "order_payment_details": []
  }
}
```

5. Checkout & Payment (Razorpay) requires token

Live and tested. This flow was verified end-to-end against a real shop's Razorpay test-mode account: checkout creates a genuine Razorpay order, a correctly-signed webhook was replayed to confirm the full order-creation path, a duplicate webhook delivery was confirmed to *not* create a second order, and a tampered signature was confirmed to be rejected.

This replaces the plain orders/store endpoint for any shop that has a payment gateway configured (see Admin Configuration below). For a shop with **no** gateway configured, orders/store still works exactly as documented in section 4 — unpaid, no gateway involved.

Step 1 — Initiate checkout

```
POST /users/{company}/orders/checkout
```

Same cart shape as orders/store (see section 4). Does **not** create an Order yet — only a pending checkout record and a real Razorpay order.

```
// Sample request
POST /users/vasantham/orders/checkout
Authorization: Bearer {token}
{
  "customer": 3547,
  "discount": 0,
  "cart": [
    { "product_id": 9, "qty": 1, "price": 5500, "tax_amount": 0 }
  ],
  "billing_customer": {
    "billing_name": "Ravi Kumar",
```

```

    "billing_phone": "9876543210",
    "billing_address": "12 Anna Salai, Chennai",
    "billing_pincode": "600001"
  }
}

// Success (200) – actual response from a real test-mode call
{
  "code": 200,
  "message": "Checkout initiated. Open Razorpay Checkout with these details to complete payment.",
  "success": true,
  "data": {
    "razorpay_order_id": "order_TccQKSJNp85zL2",
    "razorpay_key_id": "rzp_test_TcbavRtMLn8o7p",
    "amount": 5500,
    "currency": "INR"
  }
}

// If the shop has no gateway configured (400)
{
  "code": 400,
  "message": "Payment gateway not configured",
  "success": false,
  "data": null,
  "errors": "Online payment is not set up for this shop yet."
}

```

Hand `razorpay_order_id` and `razorpay_key_id` straight to the Razorpay Checkout SDK (Android SDK or Checkout.js, depending on the client) — that's what actually opens the payment UI to the customer. The API does not do that part; it only creates the order for the SDK to use.

Step 2 — Webhook (server-to-server, not called by your client)

POST `/users/{company}/webhooks/razorpay`

This endpoint is called **by Razorpay's servers**, not by your app/website. Each shop registers this exact URL (with their own `{company}` slug) in their own Razorpay Dashboard → Webhooks, along with a webhook secret they choose — the same secret entered in the admin panel's Payment Gateway settings.

Required header: `X-Razorpay-Signature` — an HMAC-SHA256 signature of the raw request body, computed using the shop's webhook secret. Requests with a missing or incorrect signature are rejected with `400` and nothing is created. This endpoint is idempotent — Razorpay may deliver the same event more than once; a duplicate delivery is safely ignored, not double-processed.

On a verified `payment.captured` or `order.paid` event, this is where the real Order gets created — `OrderDetail` rows, a payment record against the **"Razorpay"** payment mode (with the gateway's payment ID stored against it), and stock is decremented for each cart item. Other event types are acknowledged but ignored.

```

// Response shapes (all HTTP 200 except signature failure)
{ "status": "ok" } // order created successfully
{ "status": "already processed" } // duplicate webhook delivery, safely ignored
{ "status": "ignored" } // event type we don't act on
{ "status": "invalid signature" } // HTTP 400 – signature didn't match

```

Order status on creation: a gateway-confirmed order is created with `status: 1` ("Approved"), not `0` ("Placed") — since Razorpay confirming payment already is the approval step a shop admin would otherwise do by hand for a direct/COD order. If you're building

a client that reads status from `orders/view/{id}` to show the customer an order timeline, a gateway-paid order will skip straight to "Approved" and never pass through "Placed".

End-to-End Sample Flow

1. **POST** `/users/{company}/customers/register` — create the account.
2. **POST** `/users/{company}/customers/login` — get `auth_token`.
3. **GET** `/users/{company}/categories` → `/sub_categories?category=` → `/products?product=` — browse the catalog (no token needed, but works fine with one attached too).
- 4a. **If the shop has a payment gateway configured:** **POST** `/orders/checkout` → open Razorpay Checkout with the returned IDs → customer pays → Razorpay's webhook creates the real order automatically. Nothing further for the client to call to "finish" the order.
- 4b. **If the shop has no gateway configured:** **POST** `/orders/store` directly, as before — order is created immediately, unpaid.
5. **GET** `/users/{company}/orders/list` or `/orders/view/{id}` — track it, either way.

Payment Gateway — Admin Configuration

Each shop's Razorpay/PayU/Stripe keys are entered by the Super Admin, per shop, on that shop's Create/Edit screen in the admin panel — not through this API. Secrets are encrypted at rest and never returned in any response, here or anywhere else. Branches share their parent shop's configuration; there is no separate per-branch gateway setup.

Only Razorpay is actually implemented right now. PayU and Stripe are selectable in the admin dropdown for future use, but a shop configured with either will get a clean 400 "Unsupported gateway" from `orders/checkout` rather than anything actually working.

Known Gaps / Things to Confirm With the Backend Team

- The plain `orders/store` endpoint (used for shops with no gateway configured) still does not record a payment method or decrement stock — that was a deliberate scope decision, not an oversight: those shops have no online payment concept yet, so there's nothing to reconcile stock/payment against automatically. Confirm whether that's acceptable long-term or whether "COD" needs its own explicit handling.
- A shop must have an active Bill Setup (online order numbering prefix) configured before any online order — via gateway or direct — can be created, or both `orders/store` and the checkout webhook will fail with "no active bill setup."